

The Complete Two Pointers Roadmap (Beginner → Advanced)

A pattern-first curriculum, not a random list. Work top to bottom — each section builds on the templates from the one before it. Difficulty tags: ● Easy · ● Medium · ● Hard.

0. Core Templates (learn these first, cold)

Opposite-direction pointers

```
left = 0
right = n - 1
while left < right:
    if condition:
        left += 1
    else:
        right -= 1
```

Same-direction pointers (read/write)

```
slow = 0
for fast in range(n):
    if keep(arr[fast]):
        arr[slow] = arr[fast]
        slow += 1
```

Fast & slow (cycle detection / find middle)

```
slow = head
fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
```

Master these three loops and almost everything below becomes "which template + which condition."

1. Opposite Direction / Left-Right Pointers

The pointers start at the two ends and converge inward.

#	Problem	LC #	Diff	Key Insight
1	Reverse String	344	●	Purest form — swap and converge
2	Valid Palindrome	125	●	Skip non-alphanumeric while comparing
3	Valid Palindrome II	680	●	Allow exactly one mismatch/skip
4	Reverse Vowels of a String	345	●	Conditional swap
5	Squares of a Sorted Array	977	●	Merge-from-ends into a new array

2. Same Direction / Fast-Slow Pointers (Arrays)

One pointer reads everything, the other only advances when it writes.

#	Problem	LC #	Diff	Key Insight
1	Move Zeroes	283	●	Write pointer only advances on non-zero
2	Remove Duplicates from Sorted Array	26	●	Write pointer skips repeats
3	Remove Element	27	●	Write pointer skips target value
4	Remove Duplicates from Sorted Array II	80	●	Write pointer allows up to 2 dups
5	Merge Sorted Array	88	●	Fill from the back to avoid overwrite

3. Two Pointers on Sorted Arrays

Why sorting turns an $O(n^2)$ search into $O(n)$.

#	Problem	LC #	Diff	Key Insight
1	Two Sum II - Input Array Is Sorted	167	●	THE archetype — sum vs target decides direction
2	Two Sum	1	●	Contrast: unsorted → HashMap, not Two Pointers
3	Sum of Square Numbers	633	●	Search a value-range with two pointers
4	Two Sum Less Than K	1099	●	Maximize instead of match exactly

4. Two Pointers + Sorting (kSum family)

Sort → fix one element → run Two Pointers on the rest.

#	Problem	LC #	Diff	Key Insight
1	3Sum	15	●	Fix smallest, two-pointer the rest, skip dupes
2	3Sum Closest	16	●	Track closest sum instead of exact match
3	4Sum	18	●	Same idea, one more fixed loop
4	3Sum Smaller 9	259	●	Counting variant (" $<$ " instead of " $==$ ")
5	3Sum With Multiplicity	923	●	Counting with repeated values
6	Valid Triangle Number	611	●	Reversed condition ($a+b>c$), counting
7	Number of Subsequences That Satisfy the Given Sum Condition	1498	●	Two pointers + power-of-2 counting

5. Partitioning / In-Place Rearrangement

#	Problem	LC #	Diff	Key Insight
1	Sort Array By Parity	905	●	Simple two-way partition
2	Sort Array By Parity II	922	●	Indexed two-way partition
3	Sort Colors	75	●	Dutch National Flag — three pointers
4	Wiggle Sort 9	280	●	Alternating in-place invariant

6. String Two Pointers

#	Problem	LC #	Diff	Key Insight
1	Is Subsequence	392	●	Same-direction, one pointer per string
2	Reverse Only Letters	917	●	Opposite pointers + skip condition
3	Add Strings	415	●	Opposite direction from the end + carry
4	Longest Palindromic Substring	5	●	Expand-around-center (pointers move outward)

7. Fast & Slow Pointers

Different speeds converge inside a cycle, or find the midpoint.

#	Problem	LC #	Diff	Key Insight
1	Middle of the Linked List	876	●	Fast moves 2x, lands slow at the middle
2	Linked List Cycle	141	●	THE archetype — fast laps slow in a cycle
3	Linked List Cycle II	142	●	Find entry point (reset + move at speed 1)
4	Palindrome Linked List	234	●	Find mid + reverse + compare
5	Happy Number	202	●	Cycle detection on an implicit sequence
6	Find the Duplicate Number	287	●	Array-as-linked-list cycle trick

8. Two Pointers on Linked Lists

#	Problem	LC #	Diff	Key Insight
1	Remove Nth Node From End of List	19	●	Gap-of-N pointers
2	Intersection of Two Linked Lists	160	●	Switch heads to equalize remaining distance
3	Merge Two Sorted Lists	21	●	One pointer per list, advance the smaller
4	Reorder List	143	●	Find-mid + reverse + merge, all combined

9. Two Pointers + Sliding Window

#	Problem	LC #	Diff	Key Insight
1	Longest Substring Without Repeating Characters	3	●	Right expands, left shrinks on repeat
2	Minimum Size Subarray Sum	209	●	Shrink while sum stays valid
3	Max Consecutive Ones III	1004	●	Shrink on exceeding a flip budget
4	Longest Repeating Character Replacement	424	●	Shrink on exceeding a replace budget
5	Minimum Window Substring	76	●	Two pointers + frequency map (need/have)

10. Two Pointers + Greedy

Prove one pointer is "safe" to move, then move it.

#	Problem	LC #	Diff	Key Insight
1	Container With Most Water	11	●	Move the shorter wall — it's the only one that can improve
2	Assign Cookies	455	●●	Sort both, greedily match
3	Bag of Tokens	948	●	Sort + opposite pointers + greedy choice
4	Boats to Save People	881	●	Sort + pair heaviest with lightest
5	Trapping Rain Water	42	●●●	Track running max from the smaller side

11. Two Pointers + Binary Search / Other Patterns

#	Problem	LC #	Diff	Key Insight
1	Max Number of K-Sum Pairs	1679	●	Two pointers (or HashMap) — compare both
2	K-diff Pairs in an Array	532	●	Sorted two pointers or HashSet
3	Count the Number of Fair Pairs	2563	●	Two pointers + binary search for counting
4	Minimize Maximum Pair Sum	1877	●	Sort + opposite pointers + greedy pairing

12. Advanced / Hard Two Pointer Problems

#	Problem	LC #	Diff	Key Insight
1	Trapping Rain Water	42	⦿	Revisit as the hardest greedy-pointer capstone
2	Minimum Window Substring	76	⦿	Revisit — hardest window+frequency-map combo
3	Longest Mountain in Array	845	⦿	Expand outward from a detected peak
4	Substring with Concatenation of All Words	30	⦿	Fixed-width word blocks + frequency map

Suggested Study Order (Phases)

1. **Phase 1 — Fundamentals (Week 1):** Section 1 (Opposite Direction) and Section 2 (Same Direction) fully. Get the read/write pointer distinction automatic.
2. **Phase 2 — Sorted-Array Fluency (Week 1–2):** Section 3, then Section 4 (kSum family). Solve 15 (3Sum) until you can write it blind, including duplicate-skipping.
3. **Phase 3 — Partitioning & Strings (Week 2):** Sections 5 & 6. Drill Dutch National Flag (75) — the three-pointer partition shows up in disguised forms later.
4. **Phase 4 — Fast & Slow (Week 3):** Section 7. Learn cycle detection cold (141), then the entry-point trick (142). Notice 202 and 287 are the "same idea, no linked list in sight" surprise problems.
5. **Phase 5 — Linked Lists (Week 3–4):** Section 8. Chain skills together in 143 (Reorder List).
6. **Phase 6 — Window & Greedy (Week 4–5):** Sections 9 & 10.

Master 11 (Container With Most Water) — the "which pointer is safe to move" proof recurs constantly.

7. **Phase 7 — Hybrid & Hard (Week 5–6):** Sections 11 & 12, timed, no hints.

Notes on Scope

LeetCode's "Two Pointers" tag is noisy — it overlaps heavily with Sliding Window, Sorting, Linked List, and Greedy tags, and includes problems where Two Pointers is only one of several valid approaches. This roadmap includes 56 curated problems where Two Pointers is a *major or intended* part of the solution, deliberately including a few contrast cases (like LC 1, Two Sum) so you learn to recognize when a HashMap beats Two Pointers rather than reaching for it by default — that judgment call is itself an interview skill.

🔒 = LeetCode Premium (1099, 259, 280) — if you don't subscribe, read the approach rather than submit.